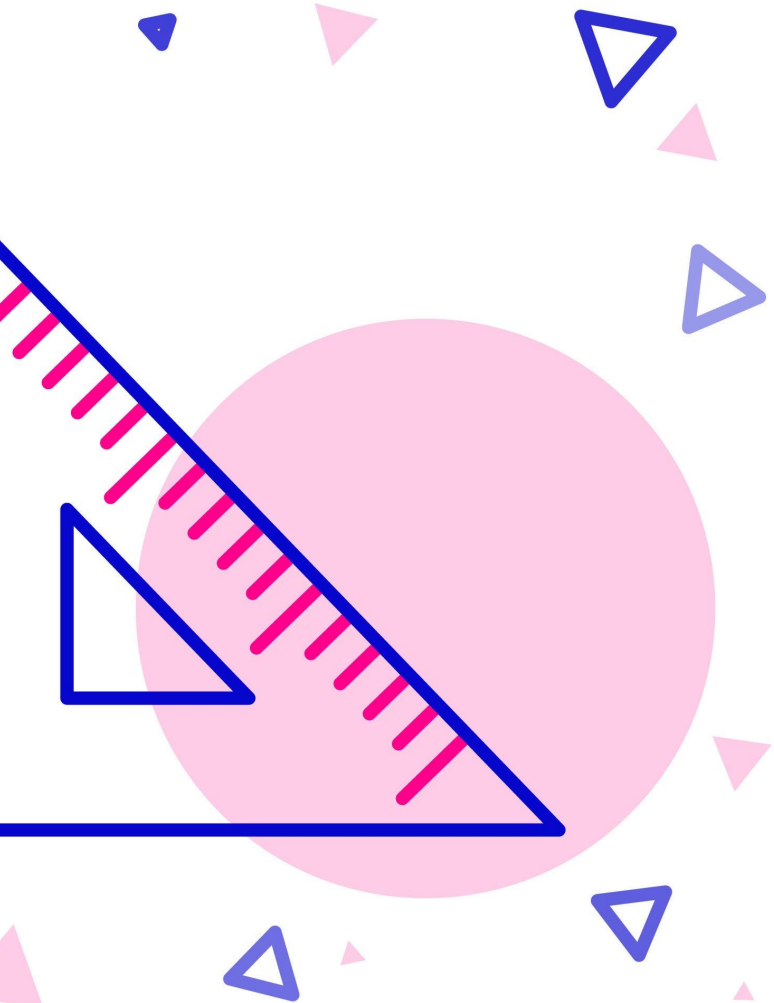


Магія самоподібності

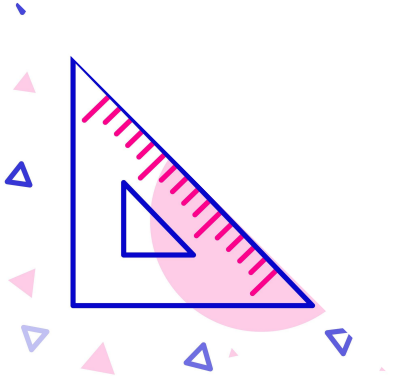
Turtle + Math + Art

Математичний фундамент: Y-дерево

Геометрія кутів: Для симетричного дерева ми використовуємо поворот ліворуч і праворуч на однаковий кут.

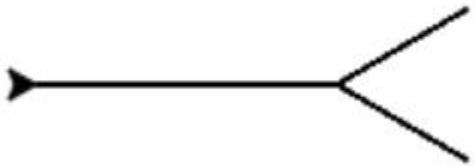
Пропорції: Кожна наступна гілка має бути коротшою за попередню (наприклад, L = L * 0.7

Вектори: Кожна гілка — це вектор, що має довжину та напрямок.



Малюємо одну гілку

```
import turtle
t = turtle.Turtle()
def branch(size):
    t.forward(size)      # Стовбур
    t.left(30)           # Гілка ліворуч
    t.forward(size/2)
    t.backward(size/2)
    t.right(60)          # Гілка праворуч
    t.forward(size/2)
    t.backward(size/2)
    t.left(30)           # Повернення в центр
    t.backward(size)
branch(100)
turtle.done()
```

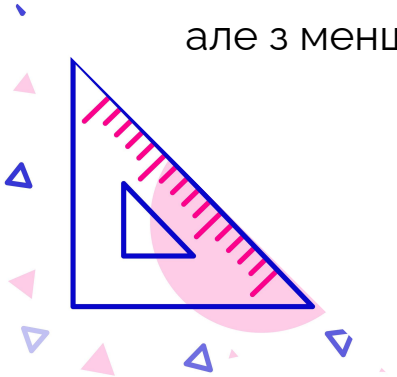


Проблема росту

Уявіть, що на кінці кожної маленької гілочки має вирости ще одна така сама "рогатка", але менша.

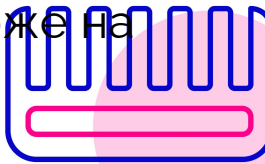
Якщо писати код вручну для 5-10 рівнів — ми втомимося.

Рішення: Функція має викликати саму себе, але з меншим розміром.



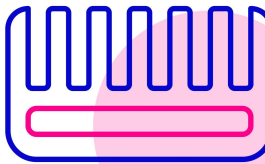
Додаємо у функцію

1. **Умову зупинки (if):** Це «гальма» для Черепашки. Без них вона намагатиметься малювати нескінченно малі гілки, і програма зависне.
   `if size < 5: return`
2. **Виклики функції всередині самої себе:** Замість того, щоб малювати лінію один раз, ми кажемо функції: «а тепер на цьому місці намалюй ще одне таке саме дерево, але менше».
   Замість **t.forward(size/2)** тепер викликаємо саму функцію
   tree(size * 0.7)
3. З**міну параметрів:** При кожному наступному кроці ми зменшуємо розмір гілки (наприклад, множимо на 0.7), щоб дерево було схоже на справжнє.



Магія рекурсії

```
import turtle
t = turtle.Turtle()
t.speed(0)          # Найшвидший режим малювання
t.left(90)          # Повертаємо Черепашку вгору (стовбур росте вгору)
t.penup()
t.goto(0, -200)     # Переносимо в нижню частину екрана
t.pendown()
def tree(size):
    if size < 5:
        return
    t.forward(size) # Малюємо поточну гілку
    t.right(30)     # Повертаємо праворуч на 30 градусів
    tree(size * 0.7) # Викликаємо функцію для малювання меншої гілки
    t.left(60)      # Повертаємо ліворуч на 60
    tree(size * 0.7) # Викликаємо функцію для малювання меншої гілки
    t.right(30)     # Повертаємось у пряме положення
    t.backward(size) # Повертаємось у початкову точку гілки
tree(100)
turtle.done()
```



Практика

Завдання «Садівник»

1. Додайте колір: Змініть колір «гілок» на коричневий, а на кінцях додайте зелені точки (t.dot).
2. Змініть розміри: Зробіть так, щоб кущ мав 3 гілки замість
3. Випадковість: Використовуйте random.randint, щоб кожна гілка мала трохи інший кут повороту.

